

AI System to Automatically Review and Summarize Research Papers

Final Project Report
Manohar Jatla

1.Objective

The objective of this project is to design and implement a fully automated system capable of generating structured academic literature reviews from research papers. The system eliminates the manual burden of searching, reading, comparing, and drafting research reviews by integrating automated retrieval, semantic processing, structured synthesis, and revision mechanisms into a single unified pipeline.

The system accepts a research topic as input and produces a complete structured review consisting of:

- Abstract
- Methods Comparison
- Results Synthesis
- APA-formatted References
- Critique and Revised Version (optional)

The architecture follows a graph-based workflow ensuring reliable and stateful progression across all stages of the review process.

2. Methodology

The system is divided into four major operational phases:

2.1 Research Phase

- Topic input through CLI or Gradio UI
- Automated search via Semantic Scholar API
- Smart paper selection (configurable limit)
- Automatic PDF download
- Metadata storage for reuse and caching

To avoid redundant API calls, the system checks existing metadata before performing a new search.

2.2 Analysis Phase

- PDF text extraction using PyMuPDF
- Text normalization
- Semantic sectioning using a Gemini-based LLM
- Validation of extracted sections
- Key findings extraction
- Cross-paper comparison (if multiple papers exist)

For a single-paper case, cross-comparison is skipped automatically.

All section data is stored as structured JSON for downstream tasks.

2.3 Writing Phase

Using GPT-based models (Gemini API), the system generates structured review sections:

- Abstract
- Methods Comparison
- Results Synthesis
- APA 7th Edition formatted references

Each section is generated using controlled prompts to maintain academic tone, structural consistency, and strict content adherence.

Outputs are saved as:

- Analytical Draft
- Final Structured Review

2.4 Review Phase

A two-stage refinement cycle is implemented:

1. Critique Generation
 - Evaluates clarity, coherence, logical flow, repetition, tone, and structure
 - Returns structured improvement suggestions
2. Revision Generation
 - Uses critique feedback
 - Produces an improved version
 - Maintains original meaning without adding external content

This creates a lightweight but professional review-refinement loop.

3. Modules

3.1 Paper Retrieval Module

Handles:

- Semantic Scholar API search
- Paper selection
- PDF download
- Metadata management

Includes duplicate detection to prevent re-downloading the same topic.

3.2 Text Extraction Module

Uses PyMuPDF to:

- Extract raw text page-by-page
- Handle layout variations
- Convert PDF into structured text format

3.3 LangGraph Processing Module

Implements a graph-based architecture:

Nodes:

- Extraction
- Normalization
- Semantic Sectioning

State:

- Maintains shared data across nodes

This ensures controlled execution order without tightly coupling components.

3.4 Analysis Module

Responsible for:

- Section validation

- Key findings extraction
- Cross-paper comparison
- Analytical draft generation

Ensures incomplete or invalid papers do not affect synthesis.

3.5 Draft Generation Module

Generates:

- Abstract
- Methods comparison
- Results synthesis
- APA references

Uses structured prompting for academic formatting.

3.6 Review Module

Implements:

- Quality evaluation
- Critique generation
- Revision generation
- Final review storage

Provides user-triggered refinement via UI buttons.

4. System Design

The system follows a graph-based architecture to maintain stateful and modular control across stages.

Search → Download → Extraction → Analysis → Writing → Revision → Final Draft

Key Characteristics:

- Directed execution flow
- Modular node-based design
- Reusable intermediate outputs
- Metadata-driven caching
- Separation of concerns across modules

The architecture ensures reliability, scalability, and easier debugging.

5. Gradio UI Integration

A professional dark-themed Gradio interface was developed for full system interaction.

Features include:

- Topic input field
- Run Full Pipeline button
- Live pipeline status window
- Final structured review display
- Critique generation button
- Revision generation button

The UI integrates:

- White-on-black minimalist design
- Interaction effects
- Real-time pipeline logging
- Structured markdown rendering

The UI displays:

- Abstract
- Methods Comparison
- Results Synthesis
- APA References

This fulfils the requirement of presenting all generated sections clearly.

6. Intelligent Reuse Mechanism

To improve efficiency and reduce API usage:

- Metadata is checked before search
- Section files are checked before running LangGraph
- Analytical drafts are checked before re-analysis
- Final structured reviews are reused if already generated

This avoids redundant computation and duplicate entries.

7. Technology Stack

Programming Language:

- Python 3x

Core Libraries:

- PyMuPDF
- LangChain
- LangGraph
- Gemini API (Google Generative AI)
- Gradio
- Pandas
- Dotenv

Architecture:

- Directed Graph Pipeline
- Modular functional design

8. Milestone Implementation Summary

Milestone 1

- Environment setup
- Semantic Scholar API integration
- PDF download automation
- Metadata preparation

Milestone 2

- PDF extraction
- Semantic sectioning
- Validation logic
- Key findings extraction
- Cross-paper comparison

Milestone 3

- Structured section generation
- Abstract, Methods, Results implementation
- Multi-paper synthesis

- APA formatting

Milestone 4

- Review and refinement cycle
- Quality evaluation module
- Polished Gradio UI
- Critique/Revise controls
- Final integrated pipeline
- Testing and documentation

9. Results and System Capabilities

The completed system is capable of:

- Fully automated literature review generation
- Multi-paper analytical synthesis
- Academic-structured output formatting
- APA-compliant reference generation
- Iterative refinement via critique-revision cycle
- UI-based real-time interaction

The system significantly reduces manual effort in literature review preparation and demonstrates the feasibility of LLM-driven academic synthesis workflows.

10. Conclusion

This project successfully implements a graph-based, LLM-powered automated research review system. By combining structured pipeline orchestration, semantic reasoning, academic writing control, and UI integration, the system achieves end-to-end automation of literature review generation.

The modular architecture ensures scalability and extensibility, while the review-refinement cycle enhances output quality. The system demonstrates how modern language models can be integrated into structured academic workflows responsibly and effectively.